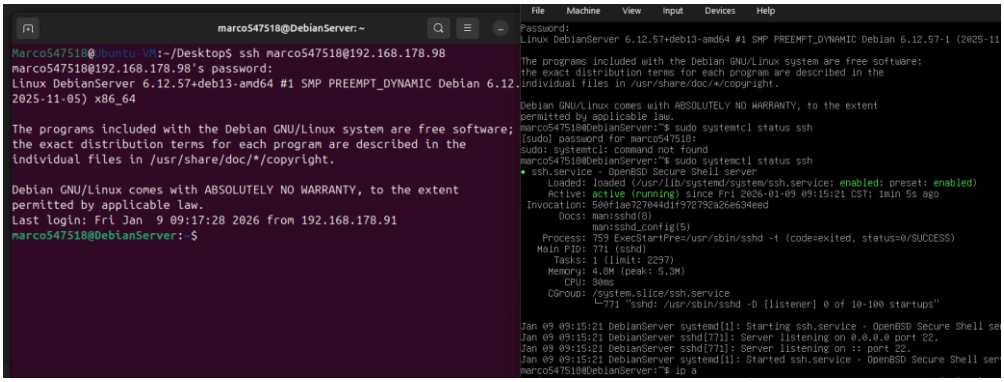
Template Week 6 – Networking

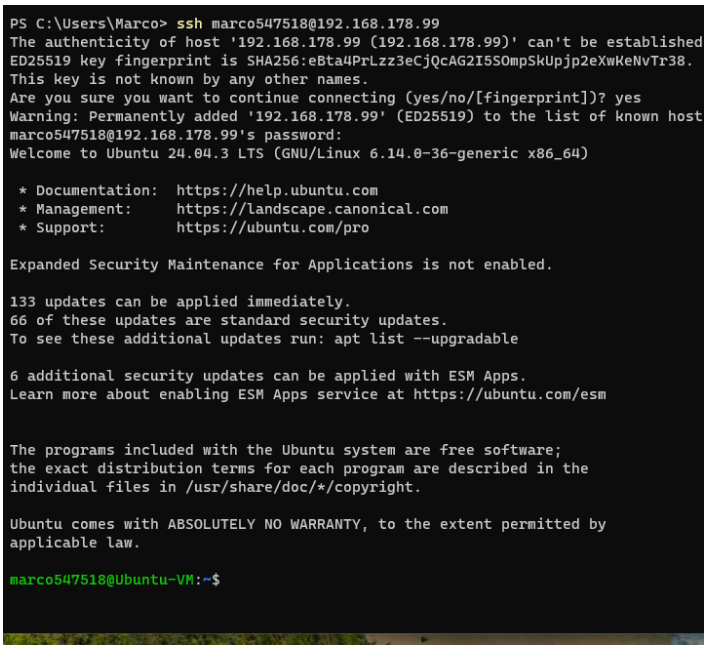
Student number: 547518

Assignment 6.1: Working from home

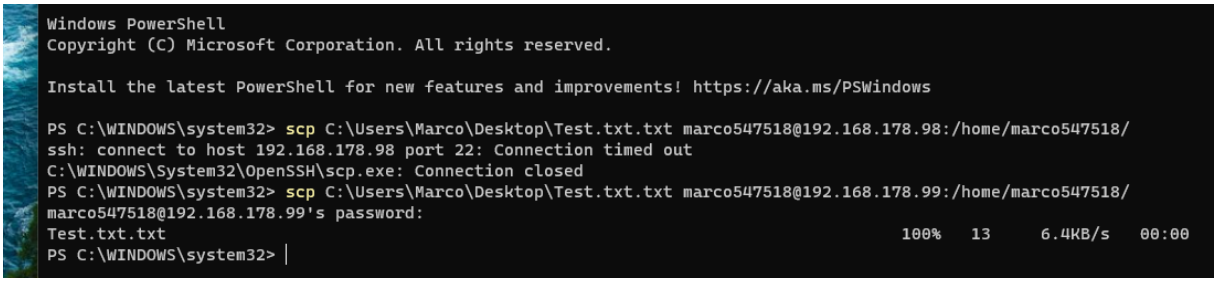
Screenshot installation openssh-server:



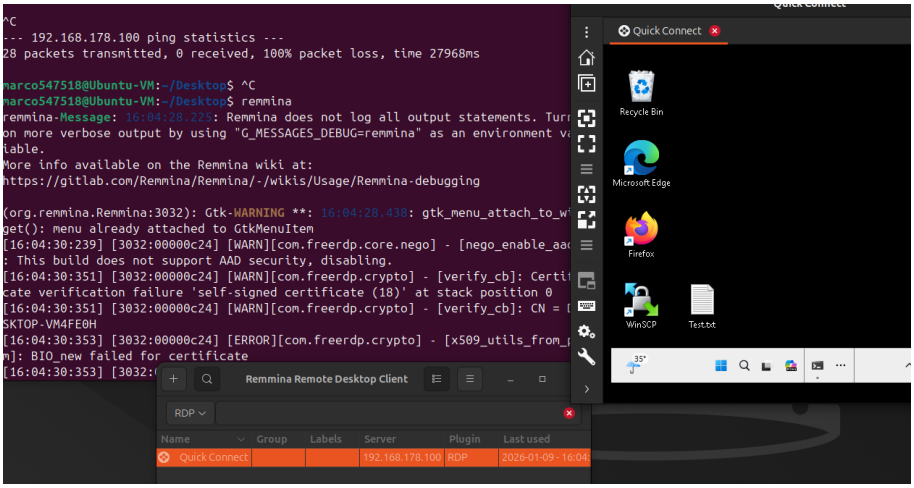
Screenshot successful SSH command execution:



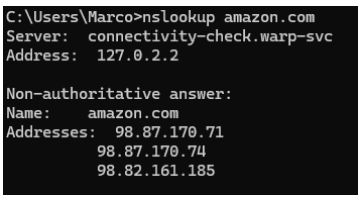
Screenshot successful execution SCP command:



Screenshot remmina:



Assignment 6.2: IP addresses websites



Relevant screenshots nslookup command:

amazon.com : 98.87.170.71

google.com : 2a00:1450:400e:803::200e and 172.217.168.238

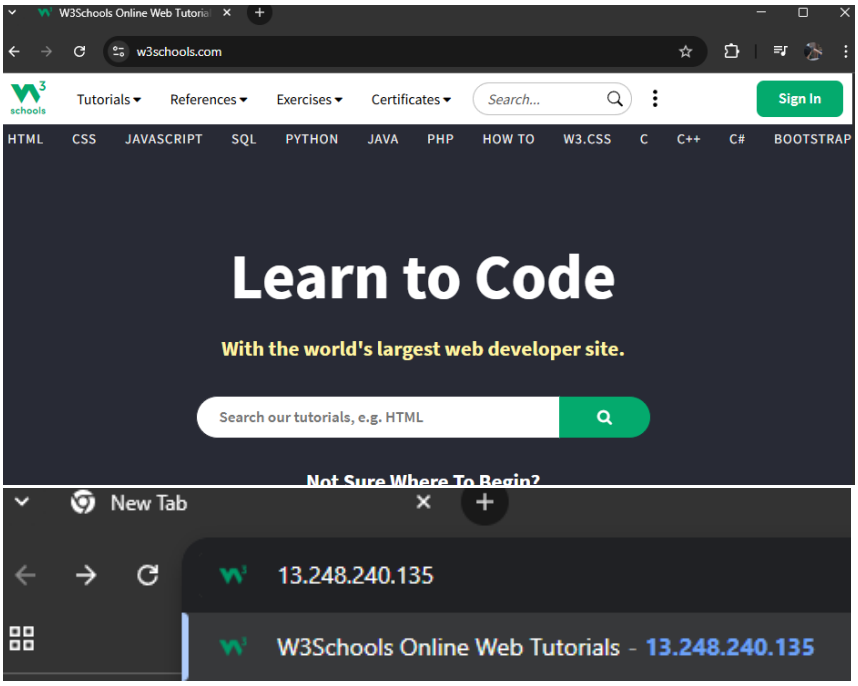
one.one.one.one: Addresses: 1.0.0.1 and 1.1.1.1

dns.google.com: 8.8.8.8 and 8.8.4.4

bol.com: 79.170.100.62

w3schools.com: 76.223.115.82 and 13.248.240.135

Screenshot website visit via IP address:



Assignment 6.3: subnetting

How many IP addresses are in this network configuration 192.168.110.128/25?

IPv4 has 32bits number of host bits = 32 – 25 = 7 bits
number of Ips = 2^7 = 128 so there are a total of 128 IP addresses

What is the usable IP range to hand out to the connected computers?

So the number of IPs are 128 which is the last one + another 127 is a total of 128
so 192.168.110.128 +127 = 192.168.110.255 is the broadcast
The usable Ips are 192.168.110.129 until 192.168.110.254 with a ***total of 126 total usable IPs***

Check your two previous answers with this Linux command: `ipcalc 192.168.110.128/25`

```
marco547518@Ubuntu-VM:~/Desktop$ ipcalc 192.168.110.128/25
Address:   192.168.110.128      11000000.10101000.01101110.1 0000000
Netmask:   255.255.255.128 = 25 11111111.11111111.11111111.1 0000000
Wildcard:  0.0.0.127            00000000.00000000.00000000.0 1111111
=>
Network:   192.168.110.128/25   11000000.10101000.01101110.1 0000000
HostMin:   192.168.110.129      11000000.10101000.01101110.1 0000001
HostMax:   192.168.110.254      11000000.10101000.01101110.1 1111110
Broadcast: 192.168.110.255      11000000.10101000.01101110.1 1111111
Hosts/Net: 126                   Class C, Private Internet
```

Explain the above calculation in your own words.

Assignment 6.4: HTML

Screenshot IP address Ubuntu VM:

```
inet 192.168.178.99/24 brd 192.168.178.255
    valid_lft 3355sec preferred_lft 3355sec
```

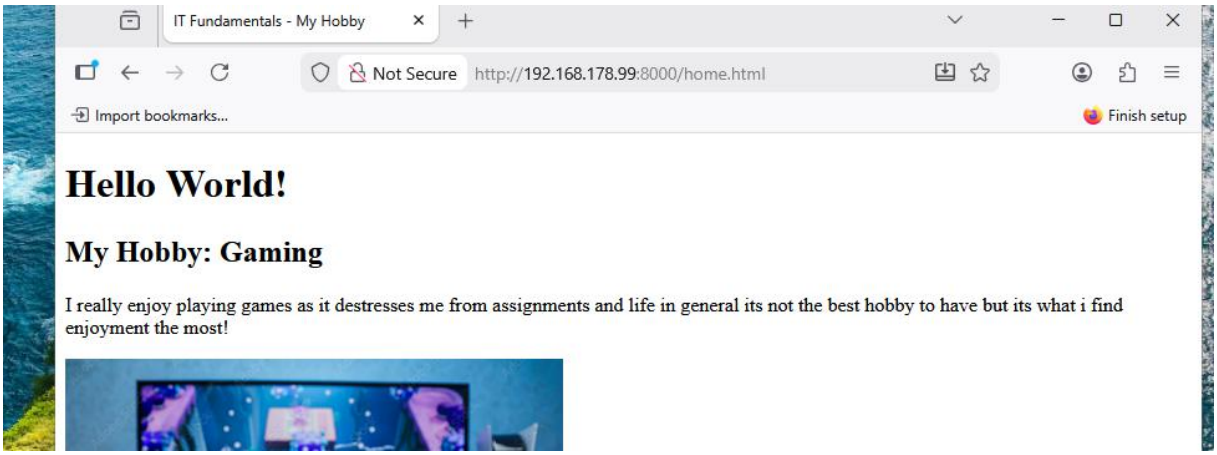
Screenshot of Site directory contents:

```
marco547518@Ubuntu-VM:~/site$ ls
css         images      pdf         week2.html  week4.html  week6.html
home.html   index.html  week1.html  week3.html  week5.html  week7.html
marco547518@Ubuntu-VM:~/site$
```

Screenshot python3 webserver command:

```
marco547518@Ubuntu-VM:~/site$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Screenshot web browser visits your site



Assignment 6.5: Network segment

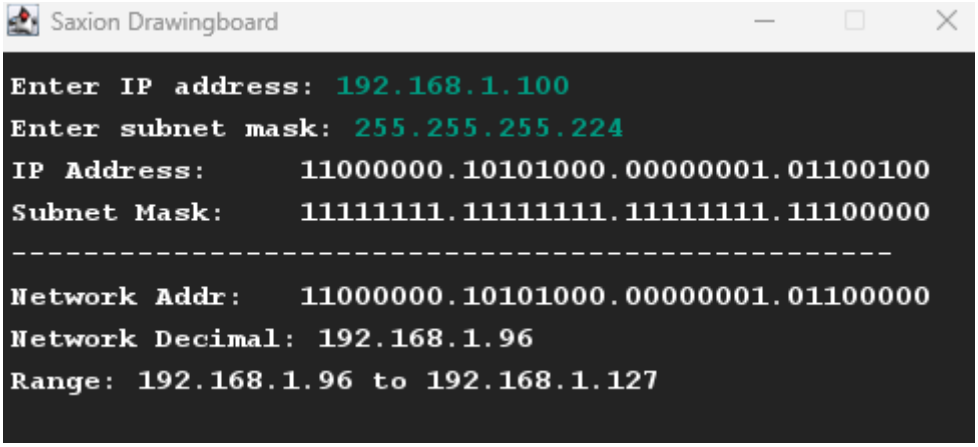
Remember that bitwise java application you've made in week 2? Expand that application so that you can also calculate a network segment as explained in the PowerPoint slides of week 6. Use the bitwise & AND operator. You need to be able to input two Strings. An IP address and a subnet.

IP: 192.168.1.100 and subnet: 255.255.255.224 for /27

```
Example: 192.168.1.100/27
Calculate the network segment
IP Address:   11000000.10101000.00000001.01100100
Subnet Mask:  11111111.11111111.11111111.11100000
-----------------------------------------------
Network Addr: 11000000.10101000.00000001.01100000

This gives 192.168.1.96 in decimal as the network address.
For a /27 subnet, each segment (or subnet) has 32 IP addresses (2^5).
The range of this network segment is from 192.168.1.96 to 192.168.1.127.
```

Paste source code here, with a screenshot of a working application.



Source code:

```
import nl.saxion.app.SaxionApp;

public class Application1 implements Runnable {

   public static void main(String[] args) {
      SaxionApp.start(new Application1(), 500, 500);
   }

   @Override
   public void run() {
      SaxionApp.print("Enter IP address: ");
      String ip = SaxionApp.readString();

      SaxionApp.print("Enter subnet mask: ");
      String subnet = SaxionApp.readString();

      int[] ipOctets = parseIp(ip);
      int[] subnetOctets = parseIp(subnet);

      SaxionApp.printLine("IP Address:     " + toBinaryString(ipOctets));
      SaxionApp.printLine("Subnet Mask:    " + toBinaryString(subnetOctets));
      SaxionApp.printLine("-----------------------------------------------");

      int[] network = new int[4];
      for (int i = 0; i < 4; i++) {
         network[i] = ipOctets[i] & subnetOctets[i];
      }

      SaxionApp.printLine("Network Addr:   " + toBinaryString(network));
      SaxionApp.printLine("Network Decimal: " + toDecimalString(network));


      int subnetBits = countSubnetBits(subnetOctets);
      int addresses = (int)Math.pow(2, 32 - subnetBits);
```

```
        int[] lastIP = network.clone();
        lastIP[3] += addresses - 1;
        SaxionApp.printLine("Range: " + toDecimalString(network) + " to " + toDecimalString(lastIP));
    }

    private int[] parseIp(String ip) {
        String[] parts = ip.split("\\.");
        int[] octets = new int[4];
        for (int i = 0; i < 4; i++) {
            octets[i] = Integer.parseInt(parts[i]);
        }
        return octets;
    }

    private String toBinaryString(int[] octets) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 4; i++) {
            sb.append(String.format("%8s", Integer.toBinaryString(octets[i])).replace(' ', '0'));
            if (i != 3) sb.append(".");
        }
        return sb.toString();
    }

    private String toDecimalString(int[] octets) {
        return octets[0] + "." + octets[1] + "." + octets[2] + "." + octets[3];
    }

    private int countSubnetBits(int[] subnetOctets) {
        int count = 0;
        for (int i = 0; i < 4; i++) {
            int n = subnetOctets[i];
            for (int j = 0; j < 8; j++) {
                if ((n & (1 << (7 - j))) != 0) count++;
            }
        }
        return count;
    }
}
```

Ready? Save this file and export it as a pdf file with the name: **week6.pdf**